영지식증명 톺아보기 2편: 컴퓨터가 이해하는 수학

Author: 김대익
Reviewed By: —

Disclaimer: 본 글은 영지식증명의 수학적 기반인 유한체와 타원곡선을 다룹니다. 암호학 비전공자도 이해할 수 있도록 직관적 설명을 우선했습니다.

---

1. 들어가며

1편에서 영지식증명의 핵심 개념을 다뤘습니다. 산술 회로, 제약 조건, SNARK의 간결함까지. 하지만 한 가지 중요한 질문을 건너뛰었습니다:

> "컴퓨터는 어떻게 이런 수학 연산을 하는가?"

일반적인 정수 연산은 문제가 있습니다. 곱셈을 하면 숫자가 계속 커지고, 나눗셈을 하면 소수점이 생깁니다. 컴퓨터는 무한한 정밀도를 다룰 수 없습니다.

유한체(Finite Field)는 이 문제를 해결합니다. 덧셈, 뺄셈, 곱셈, 나눗셈을 모두 할 수 있으면서도, 결과가 항상 정해진 범위 안에 머무르는 수의 세계입니다.

타원곡선(Elliptic Curve)은 이 유한체 위에서 정의되며, "쉽게 계산되지만 역추적이 불가능한" 연산을 제공합니다. 이것이 공개키 암호화와 ZKP의 보안 기반입니다.

이번 편에서는:

- **왜 유한체를 쓰는가** → 컴퓨터의 한계
- **왜 소수여야 하는가** → 나눗셈(역원) 보장
- **왜 타원곡선인가** → RSA 대비 짧은 키 크기
- **ECDLP** → 단방향 함수의 핵심

---

2. 무한의 문제: 왜 유한체가 필요한가

2.1 일반 정수 연산의 한계

1편에서 `x³ + x + 5 = 35`를 예로 들었습니다. x = 3일 때 성립하죠.

하지만 실제 ZKP 회로에서는 수백만 개의 제약 조건이 있고, 각 값이 수백 비트에 달합니다. 문제는:

1. **곱셈 폭발**: 256비트 숫자 두 개를 곱하면 512비트가 됩니다. 회로를 통과할 때마다 크기가 두 배씩 늘어나면 금방 감당할 수 없는 크기가 됩니다.

2. **나눗셈 문제**: 정수에서 7 ÷ 3 = 2.333...입니다. 소수점은 무한히 이어집니다. 컴퓨터는 이걸 정확히 표현할 수 없습니다.

3. **비교 불가**: "이 값이 저 값과 같은가?"를 확인하려면 두 값의 비트 표현이 완전히 같아야 합니다. 무한 정밀도에서는 이것조차 어렵습니다.

2.2 시계 산술: 유한체의 직관

해결책은 **모듈러 연산(Modular Arithmetic)**입니다. 시계를 생각해보세요.

시계에서 10시 + 5시간 = 15시가 아니라 **3시**입니다. 12를 넘으면 다시 처음으로 돌아갑니다.

수학적으로:

```
(10 + 5) mod 12 = 3
```

이것이 "mod 12" 연산입니다. 모든 결과가 0~11 사이에 머뭅니다.

핵심 이점: 아무리 큰 숫자를 더하거나 곱해도, 결과는 항상 0부터 (p-1) 사이에 머뭅니다. 이것을 **폐쇄성(Closure)**이라고 합니다.

2.3 유한체의 정의

유한체(Finite Field) 또는 **갈루아 체(Galois Field)**는 유한한 개수의 원소로 이루어진 집합으로, 덧셈과 곱셈이 정의되어 있고 다음을 만족합니다[^1]:

1. **폐쇄성**: 연산 결과가 항상 집합 안에 있다
2. **결합법칙**: (a + b) + c = a + (b + c)
3. **교환법칙**: a + b = b + a, a × b = b × a
4. **항등원 존재**: 0(덧셈), 1(곱셈)이 존재
5. **역원 존재**: 모든 원소에 대해 덧셈 역원이, 0이 아닌 모든 원소에 대해 곱셈 역원이 존재

마지막 조건 "곱셈 역원"이 핵심입니다. 이것이 **나눗셈을 가능하게** 합니다.

[^1]: Wikipedia. *Finite field*. https://en.wikipedia.org/wiki/Finite_field

---

3. 왜 소수여야 하는가

3.1 역원이 있어야 나눗셈이 가능하다

곱셈 역원이란: a의 역원 a^{-1}은 `a × a⁻¹ = 1`을 만족하는 수입니다.

일반 정수에서 3의 곱셈 역원은? 1/3 = 0.333...이므로 정수가 아닙니다. 정수에서는 1과 -1만 곱셈 역원을 갖습니다.

하지만 mod 7에서:

```
3 × 5 = 15 = 2 × 7 + 1 = 1 (mod 7)
```

3의 역원은 5입니다! 확인: 3 × 5 = 15, 15 mod 7 = 1.

이제 "3으로 나누기"를 "5를 곱하기"로 바꿀 수 있습니다.

3.2 소수가 아니면 역원이 없을 수 있다

mod 6에서 2의 역원을 찾아봅시다:

```
2 × 1 = 2 (mod 6)
2 × 2 = 4 (mod 6)
2 × 3 = 0 (mod 6)  // 6이 되어 0으로 돌아옴
2 × 4 = 2 (mod 6)
2 × 5 = 4 (mod 6)
```

어떤 수를 곱해도 1이 나오지 않습니다. **2의 역원이 존재하지 않습니다**.

이유: 2와 6은 **공약수 2**를 갖습니다. gcd(2, 6) = 2 ≠ 1.

정리: Z_n이 체(Field)가 되려면, 모든 0이 아닌 원소가 곱셈 역원을 가져야 합니다. 이것은 **n이 소수일 때만** 보장됩니다[^2].

소수 p에서 1, 2, 3, ..., p-1 중 어떤 수도 p와 공약수를 갖지 않습니다. 따라서 모든 수가 역원을 갖습니다.

[^2]: Boone, K. *An overview of finite fields*. https://kevinboone.me/finite_fields.html

3.3 실제 ZKP에서 사용하는 소수

암호학에서 사용하는 소수는 매우 큽니다. 예를 들어 BN254 커브에서 사용하는 소수 p:

```
p = 21888242871839275222246405745257275088696311157297823662689037894645226208583
```

약 254비트, 10진수로 77자리입니다. 이 범위의 모든 수에 대해 역원이 존재하고, 모든 연산 결과가 이 범위 안에 머뭅니다.

3.4 역원 계산: 확장 유클리드 알고리즘

역원을 어떻게 계산할까요? **확장 유클리드 알고리즘(Extended Euclidean Algorithm)**을 사용합니다.

베주 항등식(Bézout's Identity):

```
gcd(a, p) = ax + py
```

p가 소수이고 a ≠ 0이면 gcd(a, p) = 1이므로:

```
ax + py = 1
ax ≡ 1 (mod p)
```

따라서 x가 a의 역원입니다.

Python 예시:

```python
def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(a, p):
    gcd, x, _ = extended_gcd(a % p, p)
    if gcd != 1:
        raise ValueError("역원이 존재하지 않습니다")
    return x % p

# 예: mod 7에서 3의 역원
print(mod_inverse(3, 7))  # 출력: 5
```

---

4. 유한체에서의 연산

4.1 기본 연산

p = 7인 유한체 F_7 = {0, 1, 2, 3, 4, 5, 6}에서:

덧셈:

```
3 + 5 = 8 mod 7 = 1
6 + 6 = 12 mod 7 = 5
```

뺄셈 (덧셈의 역원 사용):

```
3 - 5 = 3 + (-5) = 3 + 2 = 5  // -5 mod 7 = 2
```

곱셈:

```
3 × 4 = 12 mod 7 = 5
5 × 5 = 25 mod 7 = 4
```

나눗셈 (곱셈의 역원 사용):

```
5 ÷ 3 = 5 × 3⁻¹ = 5 × 5 = 25 mod 7 = 4

// 검증: 4 × 3 = 12 mod 7 = 5 ✓
```

4.2 제곱근도 존재한다

놀라운 점: 유한체에서는 "완전 제곱수"가 아닌 수도 제곱근을 가질 수 있습니다.

F_{11}에서:

```
5² = 25 mod 11 = 3
```

따라서 3의 제곱근은 5입니다! 일반 정수에서 √3은 무리수이지만, F_{11}에서는 정확히 5입니다.

단, 모든 원소가 제곱근을 갖는 것은 아닙니다. 제곱근을 갖는 원소를 **이차 잉여(Quadratic Residue)**라고 합니다.

4.3 왜 이게 중요한가

ZKP에서 다항식을 다룰 때, 덧셈, 뺄셈, 곱셈, 나눗셈이 모두 필요합니다.

- R1CS 제약 조건: `a × b = c`
- 라그랑주 보간: 나눗셈 필요
- 다항식 평가: 거듭제곱 필요

유한체는 이 모든 연산을 **정확하게** 수행할 수 있고, 결과가 항상 **고정된 크기** 안에 머뭅니다. 오버플로우도 없고, 반올림 오차도 없습니다.

---

5. 타원곡선: 왜 등장하는가

5.1 암호학의 핵심: 단방향 함수

암호학의 기본 요구사항:

> "계산하기는 쉽지만, 역산하기는 어려운 함수"

예를 들어:

- **쉬움**: 두 소수 p, q를 곱해서 n = p × q를 계산
- **어려움**: n이 주어졌을 때 p와 q를 찾기 (소인수분해)

RSA는 이 "소인수분해의 어려움"에 보안을 의존합니다.

5.2 이산 로그 문제 (DLP)

또 다른 단방향 함수: **이산 로그 문제(Discrete Logarithm Problem, DLP)**

유한체 F_p에서 생성자 g가 주어졌을 때:

- **쉬움**: x가 주어지면 y = g^x mod p 계산
- **어려움**: y가 주어졌을 때 x 찾기

예: p = 23, g = 5

```
5^7 mod 23 = 17
```

7 → 17은 쉽게 계산됩니다. 하지만 17만 보고 7을 찾는 것은 어렵습니다.

작은 수에서는 모든 경우를 시도할 수 있지만, p가 256비트라면 2^256가지 경우를 시도해야 합니다.

5.3 왜 RSA 대신 타원곡선인가

RSA와 기존 DLP도 안전하지만, **키 크기가 너무 큽니다**.

보안 수준	대칭키 (AES)	RSA 키 크기	ECC 키 크기
128비트	128비트	3072비트	**256비트**
192비트	192비트	7680비트	**384비트**
256비트	256비트	15360비트	**512비트**

동일한 보안 수준에서:

- RSA 3072비트 ≈ ECC 256비트[^3]
- **약 12배 작은 키**로 같은 보안

이 차이는 실용적으로 엄청납니다:

- 저장 공간 절약
- 전송 대역폭 절약
- 연산 속도 향상 (특히 모바일/IoT 기기)
- ZKP 증명 크기 감소

[^3]: NIST. *Recommendation for Key Management*. 및 다수 문헌에서 확인.

5.4 타원곡선이 더 안전한 이유

같은 키 크기에서 타원곡선이 더 안전한 이유:

유한체 DLP: Index Calculus 같은 준지수 시간(sub-exponential) 알고리즘이 존재합니다. 따라서 키를 크게 해야 합니다.

타원곡선 DLP (ECDLP): 알려진 가장 빠른 알고리즘은 Pollard's rho로, O(√n) 시간이 걸립니다[^4]. 256비트 커브에서는 2^128번의 연산이 필요하며, 이는 현재 기술로 불가능합니다.

[^4]: Galbraith, S. D., & Gaudry, P. (2016). *Recent progress on the elliptic curve discrete logarithm problem*. Designs, Codes and Cryptography, 78(1), 51-72.

---

6. 타원곡선의 구조

6.1 실수 위의 타원곡선 (직관용)

타원곡선의 일반적인 형태 (Weierstrass form):

```
y² = x³ + ax + b
```

여기서 $4a^3 + 27b^2 \neq 0$ (곡선이 "특이점"을 갖지 않기 위한 조건)

실수 위에서 이 곡선은 다음과 같이 생겼습니다:

```
        y
        |
        |
        |
```

6.4 유한체 위의 타원곡선

실제 암호학에서는 실수가 아닌 **유한체 F_p 위에서** 타원곡선을 정의합니다.

```
y² ≡ x³ + ax + b (mod p)
```

실수에서의 "부드러운 곡선"은 사라지고, **이산적인 점들의 집합**이 됩니다.

예: $y^2 = x^3 + x + 1$ over F_{23}

이 곡선 위의 점들:

```
(0, 1), (0, 22), (1, 7), (1, 16), (3, 10), (3, 13),
(4, 0), (5, 4), (5, 19), (6, 4), (6, 19), ...
```

각 점 (x, y)는 방정식을 만족합니다:

```
y² mod 23 = (x³ + x + 1) mod 23
```

점 덧셈의 공식은 동일하지만, 모든 연산이 mod p로 수행됩니다.

6.5 스칼라 곱 (Scalar Multiplication)

스칼라 곱은 점을 자기 자신과 반복해서 더하는 것입니다:

```
kP = P + P + P + ... + P  (k번)
```

Double-and-add 알고리즘으로 O(log k) 시간에 계산할 수 있습니다:

예: 13P 계산 ($13 = 1101_2$)

```
1P
2P = 1P + 1P
4P = 2P + 2P
8P = 4P + 4P
13P = 8P + 4P + 1P
```

불과 6번의 덧셈으로 13P를 계산했습니다. k가 256비트라도 ~256번의 덧셈으로 충분합니다.

---

7. ECDLP: 보안의 핵심

7.1 정의

타원곡선 이산 로그 문제 (Elliptic Curve Discrete Logarithm Problem, ECDLP)[^5]:

> 타원곡선 E 위의 점 G(생성자)와 P = kG가 주어졌을 때, 정수 k를 찾아라.

- **쉬움**: k와 G가 주어지면 P = kG 계산 → O(log k) 시간
- **어려움**: G와 P가 주어졌을 때 k 찾기 → O(√n) 시간 (n은 곡선의 위수)

256비트 커브에서 n ≈ 2^256이므로, √n ≈ 2^128번의 연산이 필요합니다. 현재 기술로 불가능합니다.

[^5]: Certicom Research. *The elliptic curve discrete logarithm problem*. https://www.certicom.com/content/certicom/en/52-the-elliptic-curve-discrete-logarithm-problem.html

7.2 공개키/비밀키의 수학적 기반

ECDLP가 공개키 암호화의 기반이 되는 방식:

1. **비밀키(Private Key)**: 랜덤한 정수 k를 선택
2. **공개키(Public Key)**: P = kG를 계산

공개키 P는 공개해도 됩니다. ECDLP의 어려움 때문에 P로부터 k를 알아낼 수 없습니다.

비트코인, 이더리움 모두 이 방식(secp256k1 커브)을 사용합니다.

7.3 간단한 예제

작은 숫자로 이해해봅시다.

커브: $y^2 = x^3 + 7$ over F_{23}
생성자: G = (3, 10)
위수: n = 7 (이 생성자로 만들 수 있는 점이 7개)

```
1G = (3, 10)
2G = (19, 5)
3G = (1, 5)
4G = (11, 3)
5G = (1, 18)
6G = (19, 18)
7G = O (무한원점, 다시 처음으로)
```

만약 비밀키 k = 4를 선택하면:

- 공개키: P = 4G = (11, 3)

공격자는 G = (3, 10)과 P = (11, 3)을 보고 k = 4를 찾아야 합니다.

이 작은 예제에서는 7개만 시도하면 되지만, 실제 커브에서는 2^256개를 시도해야 합니다.

7.4 왜 ZKP에서 중요한가

ECDLP는 ZKP의 여러 핵심 요소에 사용됩니다:

1. **다항식 커밋먼트**: KZG 커밋먼트에서 다항식을 "점"으로 숨김

2. **Trusted Setup**: τ의 거듭제곱을 타원곡선 점으로 인코딩
3. **영지식성**: 비밀 값을 커브 위의 점으로 변환하여 공개해도 원래 값을 알 수 없음

다음 편에서 R1CS와 다항식 변환을 다룰 때, 이 타원곡선 점들이 어떻게 사용되는지 구체적으로 볼 것입니다.

---

8. 실제 사용되는 커브들

8.1 ZKP에서 자주 쓰이는 커브

커브	사용처	특징
BN254	Ethereum (이전), Zcash Sprout	빠른 페어링, ~100비트 보안
BLS12-381	Ethereum 2.0, Zcash Sapling, Filecoin	128비트 보안, Powers of Tau
Pasta (Pallas/Vesta)	Zcash Orchard, Halo2	재귀 증명에 최적화
secp256k1	Bitcoin, Ethereum 서명	페어링 없음, ECDSA용

8.2 페어링 친화적 커브

Groth16, PLONK 등 대부분의 SNARK는 **페어링(Pairing)**이라는 특수 연산을 사용합니다.

페어링은 두 타원곡선 점을 입력받아 다른 그룹의 원소를 출력하는 함수입니다:

```
e: G₁ × G₂ → G_T
```

이 연산이 효율적으로 가능한 커브를 **페어링 친화적 커브(Pairing-friendly Curve)**라고 합니다.

BN254와 BLS12-381이 대표적입니다.

8.3 BLS12-381의 파라미터

BLS12-381은 현재 ZKP에서 가장 널리 사용되는 커브 중 하나입니다[^6]:

```
p =
0x1a0111ea397fe69a4b1ba7b6434bacd764774b84f38512bf6730d2a0f6b0f6241eabfffeb153ffff
b9feffffffffaaab
```

- 필드 크기: 381비트
- 128비트 보안 수준
- G_1 점: 96 bytes (압축 시 48 bytes)
- G_2 점: 192 bytes (압축 시 96 bytes)

Groth16 증명이 G_1 점 2개 + G_2 점 1개로 구성되어 약 192 bytes인 이유입니다.

[^6]: Bowe, S., et al. (2017). *BLS12-381: New zk-SNARK Elliptic Curve Construction*.

---

9. 마치며: 도구들의 관계

이번 편에서 다룬 것:

개념	역할	왜 필요한가
유한체	수의 범위 제한	컴퓨터가 정확히 계산 가능
소수 모듈러	모든 원소에 역원 보장	나눗셈 연산 가능
타원곡선	단방향 함수 제공	RSA보다 짧은 키로 동일 보안
ECDLP	보안의 수학적 기반	비밀키 → 공개키 계산은 쉬움, 역방향은 불가능

다음 편에서는 **산술화(Arithmetization)**를 다룹니다:

- 계산을 R1CS 제약 조건으로 변환
- **왜 행렬인가**: n개의 제약 조건을 하나로 묶음
- **왜 다항식인가**: n개의 값을 하나의 함수로 압축
- **왜 라그랑주 보간법인가**: n개의 점에서 다항식 복원
- **왜 FFT인가**: $O(n^2) \rightarrow O(n \log n)$ 최적화

유한체와 타원곡선이라는 "무대"가 준비되었으니, 이제 그 위에서 어떤 "마법"이 펼쳐지는지 볼 차례입니다.

시리즈 로드맵

1편: 비밀을 증명한다는 것 ✓
2편: 컴퓨터가 이해하는 수학 (현재) ✓
3편: 프로그램을 다항식으로 - 산술화
4편: Zcash Sprout - 최초의 프라이버시 암호화폐 (2016)
5편: Groth16의 마법 (2016)
...

References

1. Wikipedia. *Finite field*. https://en.wikipedia.org/wiki/Finite_field
2. Wikipedia. *Elliptic-curve cryptography*. https://en.wikipedia.org/wiki/Elliptic-curve_cryptography
3. RareSkills. *Finite Fields and Modular Arithmetic for ZK Proofs*. https://rareskills.io/post/finite-fields
4. Galbraith, S. D., & Gaudry, P. (2016). *Recent progress on the elliptic curve discrete logarithm problem*. Designs, Codes and Cryptography, 78(1), 51-72. https://eprint.iacr.org/2015/1022.pdf
5. Certicom Research. *The elliptic curve discrete logarithm problem*. https://www.certicom.com/content/certicom/en/52-the-elliptic-curve-discrete-logarithm-problem.html
6. Boone, K. *An overview of finite fields*. https://kevinboone.me/finite_fields.html
7. Kak, A. (2025). *Lecture 5: Finite Fields (PART 2) - Modular Arithmetic*. Purdue University. https://engineering.purdue.edu/kak/compsec/NewLectures/Lecture5.pdf